

Introduction to Solidity

Instructor: Dendej Sawarnkatat

dendej@gmail.com

Agenda

- Introduction
- Functions
- Global Available Properties
- Ethereum Development Environment

INTRODUCTION

3

Pragma

- A pragma directive of solidity always has versions in the form 0.x.0 or x .0.0.
- This ensures that any future version of the solidity compiler is ignored that might introduce incompatibility:

```
pragma solidity ^0.4.0;
```

4

Import

- Supports import statements, which are very similar to JavaScript.
- This keyword is used at contract compile time in our local build environment.
- Once a contract gets deployed on a blockchain using EVM, it can neither read nor write files.

```
import "filename";
import * as symbolName from "filename";
import {symbol1 as alias, symbol2} from "filename";
import "github.com/ethereum/dapp-bin/library/iterable_mapping.sol" as it_mapping
```

5

Contract

- Solidity is often called a contract-oriented language.
- Since every operation that can occur on the blockchain must reside inside the scope of a contract.
- Is similar to classes in object-oriented languages.
- Can inherit from other contracts.

6

Declaration

- A contract can contain declarations ranging from:
  - functions and function modifiers
  - state variables
  - events
  - Enum
  - struct types.

7

Declaration Types

Declaration	Description	Example
State Variables	Permanently stored values in contract storage	uint32,bool, ufixed,address

Functions	Executable code units inside a contract	contract SampleContract {
		function newfunction() payable {// Function
		// code
		}
		}

Function Modifiers	Used as a declarative way of amending semantics of functions	contract newContract {
		address public bidder;
		
		modifier onlyBidder() {// Modifier
		require(msg.sender == bidder);
		_;
		}
		
		function abort() onlyBidder {// Modifier usage
		// code
		}
		}

Events	Used for EVM logging facility and Convenience Interface	contract NewAuction {
		event HighBidIncrease(address bidder, uint amount); // Event
		
		function bid() payable {
		// code
		HighBidIncrease(msg.sender, msg.value); // Triggering event
		}
		}

Structs Types	Customized type to group several variables	contract VotApp {
		struct Electorate {// Struct
		uint identity;
		bool voted;
		address delegate;
		uint vote;
		}
		}

Enum Types	Customized type with a finite set of values	contract Volcano {
		enum State {Active, Extinct, Dormant }// Enum
		}

8

State Variables

- Values which are permanently stored in contract storage.
- Valid state variable type can be categorized into three groups:
  - value type
  - reference type
  - mapping

9

Value Types

- Boolean
- Integer
- Fixed point number
- Address
- Fixed-size byte array
- Dynamically-size byte array
- Literal
- Enum

10

Reference Types

- Array
- Array Literal
- Struct

11

Data Location

- An additional annotation about where the data is stored.
- There are three data locations:
  - Memory
  - Storage
  - Calldata

12

Assignment Behavior

- Assignments between storage and memory (or from calldata) always create an independent copy.
- Assignments from memory to memory only create references.
- Assignments from storage to a local storage variable also only assign a reference.

13

Mapping

- Declared as
  - ***mapping*** *(****_KeyType*** *=>* ***_ValueType****)*.
- ***_KeyType*** can be almost any type except for a mapping, a dynamically sized array, a contract, an ***enum*** and a ***struct***.
- ***_ValueType*** can actually be any type, including mappings.
- The key data is not actually stored in a mapping, only its keccak256 hash used to look up the value.

14

FUNCTIONS

15

Functions

- Solidity has basically two types of function calls.
- **Internal function** (a message call) does not make an actual call to the EVM.
- **External function call**, which is part of the contract interface and can be called from other contracts via transactions.
- An external function cannot make internal calls.

16

Function Visibility

- By default function visibility is public.
- For state variables, external is not possible and the default is internal.
- External functions are part of the contract interface, which means they can be called from other contracts and via transactions.
- Public functions are part of the contract interface and can be either called internally or via messages.

17

Function Visibility

- Internal functions and state variables can only be accessed internally without using this.
- Private functions and state variables are only visible for the contract they are defined in and not in derived contracts.

18

"*selfdestruct*" function

- Uses to terminate or deactivate a target smart contract.
- It needs an input parameter in terms of the address to send the remaining ethers
- After the killing occurs, the contract is marked as destructed on the blockchain.

19

Function Modifiers

- Can be inherited and overridden by child contracts.
- Multiple modifiers will be evaluated in order specifying in a whitespace-separated list.
- Also accepts passed arguments.
- Inside the modifier, the next modifier body or function body, whichever comes next, is inserted where ***_;*** appears.

20

Function Types

- The getter function with external visibility for all state variables are created automatically by a compiler.
- A view declaration ensures that the state of a function is not modified.
- A pure declaration ensures the function is not read and does not modify the state of a function.
- Fallbacks are basically nameless functions that generally do not have arguments and cannot return anything.

21

Function Variables

- A variable which is declared will have an initial default value whose byte-representation is all zeros.
- A variable declared anywhere within a function will be in scope for the entire function, regardless of where it is declared.

22

Internal Function Call

- Functions of the current contract can be called directly (internally), also recursively.

```
contract C {
  function g(uint a) public pure returns (uint ret) {
    return a + f();
  }
  function f() internal pure returns (uint ret) {
    return g(7) + f();
  }
}
```

23

External Function Calls

- Employ a message call instead of jump.
- Functions of other contracts have to be called externally for which all function arguments have to be copied to memory.
- We can specify the amount of Wei or gas sent with the call with ***.value()*** and ***.gas()***, respectively.
- Any Wei you send to the contract is added to the total balance of the contract.

24

Example

```
contract InfoFeed {
  function info() public payable returns (uint ret) {
    return 42;
  }
}

contract Consumer {
  InfoFeed feed;
  function setFeed(InfoFeed addr) public {
    feed = addr;
  }
  function callFeed() public {
    feed.info.value(10).gas(800)();
  }
}
```

25

Events

- EVM logging facilities can be conveniently used by a smart contract via events.
- Most DApp web apps listen for events and, in turn, initiate JavaScript calls in the DApp's user interface.
- When such event calls happen, they cause arguments to get stored in transaction logs.

26

Events

- These logs are a special data structure in the blockchain and are associated with the address of the contract.
- These logs stay as long as a block is accessible.
- We cannot access the event or log data within a contract.

27

Inheritance

- Solidity supports multiple inheritance by copying code including polymorphism.
- Even if a contract inherits from multiple other contracts, only a single contract is created on the blockchain
- The code from the parent contracts is always copied into the final contract.

28

Example

```
contract sample1 {
  function a();
  function b();
}

//sample2 inherits sample1
contract sample2 is sample1 {
  function b();
}

contract sample3 {
  function a(int b) {

    }
}
```

29

Example

```
// sample4 inherits from sample1 and sample2
// Note that sample1 is also parent of sample2, yet there is
// only a single instance of sample1
contract sample4 is sample1, sample2, sample3 {
  function a(){}
  function c(){
    //this executes the "a" method of sample3 contract
    a();
    //this executes the 'a" method of sample1 contract
    sample1.a();

    // calls sample2.b() because it's in last in the parent
    // contracts list and therefore it overrides sample1.b()
    b();
  }
}
```

30

Example

```
// If a constructor takes an argument, it needs to be
// provided at the constructor of the child contract.

// In Solidity child constructor doesn't call parent
// constructor instead parent is initialized and copied to
//
// child
contract sample5 is sample3(122) {

}
```

31

super keyword

- Used to refer to the next contract in the final inheritance chain.

```
contract sample1 {}
contract sample2 {}
contract sample3 is sample2 {}
contract sample4 is sample2 {}

contract sample5 is sample4 {
    function myFunc() {}
}

contract sample6 is sample1, sample2, sample3, sample5 {
    function myFunc() {
        //sample5.myFunc()
        super.myFunc();
    }
}
```

32

Example Explained

- The final inheritance chain with respect to the **sample6** contract is **sample6**, **sample5**, **sample4**, **sample2**, **sample3**, **sample1**.
- The inheritance chain starts with the most derived contracts and ends with the least derived contract.

33

Abstract Contracts

- Contracts that only contain the prototype of functions instead of implementation.
- Such contracts cannot be compiled.
- If a contract inherits from an abstract contract and does not implement all non-implemented functions by overriding, it will itself be abstract.

34

Using Abstract Contracts

- Abstract contracts are only provided to make the interface known to the compiler.
- Useful when referring to a deployed contract and calling its functions.

35

Example

```
contract sample1 {
  function a() returns (int b);
}

contract sample2 {
  function myFunc() {
    sample1 s =
      sample1(0xd5f9d8d94886e70b06e474c3fb14fd43e2f23970);

    //without abstract contract this wouldn't have compiled
    s.a();
  }
}
```

36

Control Structures

- Most of the control structures from JavaScript are available in Solidity except for switch and goto.
- Available structures are:
  - if, else, while, do, for, break, continue, return, ? :
- Parentheses can not be omitted for conditionals, but curly braces can be omitted around single-statement bodies.
- No type conversion from non-boolean to boolean types as there is in C and JavaScript.

37

Returning Multiple Values

- Solidity allows functions to return multiple values.

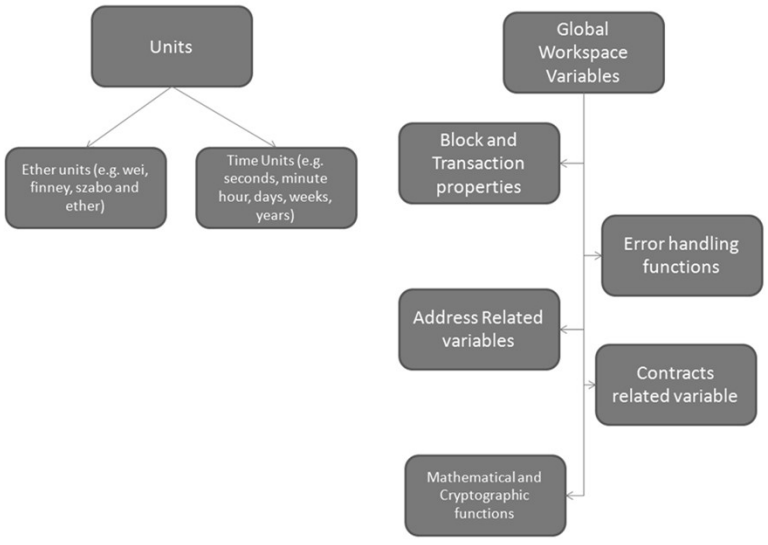
```
contract sample {
  function a() returns (int a, string c) {
    return (1, "ss");
  }
  function b() {
    int A;
    string memory B;
    //A is 1 and B is "ss"
    (A, B) = a();
    //A is 1
    (A,) = a();
    //B is "ss"
    (, B) = a();
  }
}
```

38

GLOBAL AVAILABLE PROPERTIES

39

Overview



40

Block Properties

- ***block.blockhash****(uint blockNumber)* returns (bytes32): hash of the given block (deprecated)
- ***block.coinbase*** *(address)*: current block miner's address
- ***block.difficulty*** *(uint)*: current block difficulty
- ***block.gaslimit*** *(uint)*: current block gas limit
- ***block.number*** *(uint)*: current block number
- ***block.timestamp*** *(uint)*: current block timestamp as seconds since unix epoch

41

Other Global Properties

- ***gasleft****() returns (uint256)*: remaining gas
- ***msg.data*** *(bytes)*: complete call data
- ***msg.gas*** *(uint)*: remaining gas - deprecated in version 0.4.21 and to be replaced by ***gasleft()***
- ***msg.sender*** *(address)*: sender of the message (current call)

42

Other Global Properties

- ***msg.sig*** *(bytes4)*: first four bytes of the call data (i.e. function identifier)
- ***msg.value (****uint****)***: number of **wei** sent with the message
- ***now*** *(uint)*: current block timestamp (alias for ***block.timestamp***)
- ***tx.gasprice*** *(uint)*: gas price of the transaction
- ***tx.origin*** *(address)*: sender of the transaction (full call chain)

43

Address type related

- *<address>.****balance*** *(uint256)*:
  - balance of the Address in Wei
- *<address>.****transfer****(uint256 amount)*:
  - send given amount of Wei to Address
- *<address>.****send****(uint256 amount) returns (bool)*:
  - send given amount of Wei to Address, returns false on failure.

44

Contract Related

- ***this***: The current contract, explicitly convertible to the address type.
- ***selfdestruct****(address recipient)*: Destroys the current contract, sending its funds to the given address.

45

Ether units

- A literal number can take a suffix of ***wei, finney, szabo***, or ether to convert between the sub-denominations of Ether.
- Ether currency numbers without a postfix are assumed to be ***wei***;
- E.g.: 2 Ether == 2000 ***finney*** evaluates to true.

46

Error Handling

- Solidity uses state-reverting exceptions to handle errors.
- Such an exception will undo all changes made to the state in the current call (and all its sub-calls) and also flag an error to the caller.
- Two functions namely '***assert***' and '***require***' can be used to check for conditions and throw an exception if the condition is not met.

47

Assert & Require

- The ***assert*** function should only be used to test for internal errors, and to check invariants.
- The ***require*** function should be used to ensure valid conditions, such as:
  - inputs, or contract state variables are met
  - to validate return values from calls to external contracts.

48

Example

```
contract Sharer {
  function sendHalf(address addr) public payable returns
(uint balance) {
    require(msg.value % 2 == 0, "Even value required.");
    uint balanceBeforeTransfer = this.balance;
    addr.transfer(msg.value / 2);
    // Since transfer throws an exception on failure and
    // cannot call back here, there should be no way for us
    // to still have half of the money.
    assert(this.balance == balanceBeforeTransfer -
      msg.value / 2);
    return this.balance;
  }
}
```